

Reviewer Pack — Minimal Rank of Noncommutative K-theory and Circuit Lower Bo...

Entry 9becc24d016e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 14:21:11 UTC

Minimal Rank of Noncommutative K-theory and Circuit Lower Bounds for XOR Tautologies

Entry ID: 9becc24d016e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 14:21:11 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (K-theory) **Field B** (complexity object): Complexity Theory: Circuit Complexity

Statement:

[‘For any homogeneous polynomial $f \in k[x_1, \dots, x_n]$, the rank of its noncommutative K-theoretic representation $\rho(f)$ is lower-bounded by the degree of the smallest XOR tautology that can be computed by a circuit of size $O(n^{\{2.5\}})$.’, ‘That is, $E[\rho(f)] = \Omega(\text{degree of the smallest XOR tautology for } f)$.’]

Rationale (proposer’s reasoning):

[‘Noncommutative K-theory provides a rich algebraic structure that captures certain properties of polynomials. If this conjecture holds, it would imply a direct connection between noncommutative geometric properties and circuit complexity, potentially leading to new insights in both fields.’, ‘This bridge might expose structural information about the difficulty of XOR tautologies, which is crucial for understanding computational problems.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 41e5a5dce2e0c18c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 seeds, the mean rank of noncommutative K-theoretic representations $\rho(f)$ for randomly generated homogeneous polynomials f exceeds the degree of the smallest XOR tautology by a statistically significant margin (p-value < 0.05).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncommutative K-theory' AND 'circuit complexity' AND 'lower bounds'
- 'minimal rank in noncommutative geometry' AND 'circuit lower bounds' - 'degree of XOR tautology' AND 'noncommutative K-theory representation'

Top relevant hits considered: - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori - [http://arxiv.org/abs/0705.1265v2] A noncommutative Bohnenblust-Spitzer identity for Rota-Baxter algebras solves Bogoliubov's recursion - [http://arxiv.org/abs/1411.6751v4] The noncommutative infinitesimal equivariant index formula: part II - [http://arxiv.org/abs/2602.24220v1] Comparing Classical and Quantum Variational Classifiers on the XOR Problem - [http://arxiv.org/abs/1601.00021v5] Pullbacks and nontriviality of associated noncommutative vector bundles - [http://arxiv.org/abs/2309.05856v1] Multi-Point Detection of the Powerful Gamma Ray Burst GRB221009A Propagation through the Heliosphere on October 9, 2022

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(A)
    m = len(b)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]

    # Forward elimination
    for i in range(n):
        if A[i][i] == 0:
            return None # Singular matrix, no unique solution

    # Normalize the pivot row
    pivot = Fraction(augmented_matrix[i][i])
    augmented_matrix[i] = [Fraction(x) / pivot for x in augmented_matrix[i]]

    # Eliminate below the pivot
    for j in range(i + 1, n):
```

```

        factor = augmented_matrix[j][i]
        augmented_matrix[j] = [augmented_matrix[j][k] - factor * augmented_matrix[i][k] for k

# Backward substitution
x = [0] * n
for i in range(n - 1, -1, -1):
    x[i] = augmented_matrix[i][-m]
    for j in range(i + 1, n):
        x[i] -= augmented_matrix[i][j] * x[j]

return x

def rank(matrix):
    if not matrix:
        return 0

    m = len(matrix)
    n = len(matrix[0])
    A_rref = gaussian_elimination(matrix, [0]*m)

    if A_rref is None:
        return 0

    rank = 0
    for row in A_rref:
        if any(row):
            rank += 1

    return rank

def degree_of_smallest_xor_tautology(poly):
    n = len(poly)
    for i in range(1, 2**n):
        tautology = [bool(i & (1 << j)) for j in range(n)]
        if all(poly[j] == tautology[j] for j in range(n)):
            return sum(tautology)
    return n

def random_polynomial(n, degree):
    coefficients = [random.randint(0, 1) for _ in range(degree + 1)]
    return [coefficients[i] * (x**i) for i, x in enumerate(range(n))]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Test each n with 5 different polynomials
            poly = random_polynomial(n, degree=n-1)
            rho_f = rank([poly[i] for i in range(j, j+degree_of_smallest_xor_tautology(poly)+1)])

            if rho_f is None:
                return {

```

```

        "metric_name": "rank",
        "metric_value": 0,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": "singular_matrix"
    }

    degree_smallest_xor = degree_of_smallest_xor_tautology(poly)
    total_metric_value += rho_f
    instances_tested += 1

mean_metric_value = total_metric_value / instances_tested
return {
    "metric_name": "rank",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": mean_metric_value > degree_smallest_xor,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction=1")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='degree_of_smallest_xor_tautology' first_failing={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fff666e6.py", line 116, in <module>

```

    result = run_trial(seed)
    ~~~~~^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fff666e6.py", line 86, in run_trial

```

    rho_f = rank([poly[i] for i in range(j, j+degree_of_smallest_xor_tautology(poly)+1)] for j in range(0, degree_of_smallest_xor_tautology(poly)+1))
    ~~~~~^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test without errors to verify if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12008
2	preregistration	ollama_remote	glm4:latest	0	0	5797
3	novelty	ollama_remote	glm4:latest	0	0	4877
4	novelty	ollama_remote	glm4:latest	0	0	6231
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14116
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16991
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16383
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12614
9	judge	ollama_remote	glm4:latest	0	0	8783

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97799 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9becc24d016e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9becc24d016e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9becc24d016e.tar.gz` (if generated)